MLOps Specialization Bootcamp

Module 1

Foundations of MLOps

Week 1: Why most models never reach production, and what MLOps does to solve it



Daniel Islam
Instructor, InnovatiCS MLOps Specialization



www.innovatics.ai
InnovatiCS 2024. All rights reserved
+1 (678) 209-9780

What you will be able to do



1. Explain what MLOps is and how it differs from DevOps and DataOps
2. Draw the ML lifecycle from problem framing to monitoring, with the loops back
3. Score a team on the maturity model (Levels 0 to 4) and say what the next level needs
4. Name the roles in an ML team, the hand-offs, and where technical debt builds up

Today

01

Gap and MLOps

02

Lifecycle, framing

03

Maturity model

04

Technical debt

05

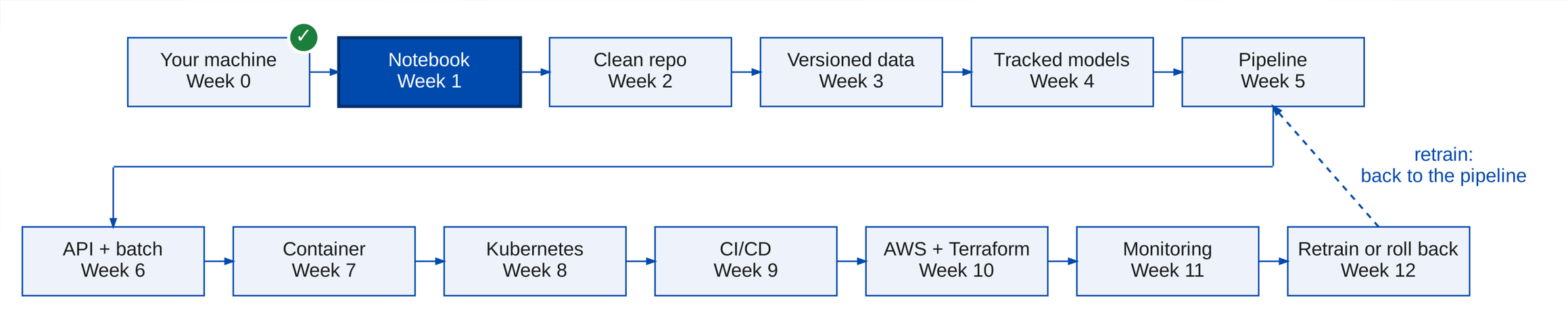
Pillars, roles, tools

06

Case, Practical 1

INNOVATICS
www.innovatics.ai
Data Science & AI From Zero to Hero

Where we are



Week 1 looks at the notebook: the "before" picture. Every later week adds one piece.

01

The production gap and MLOps

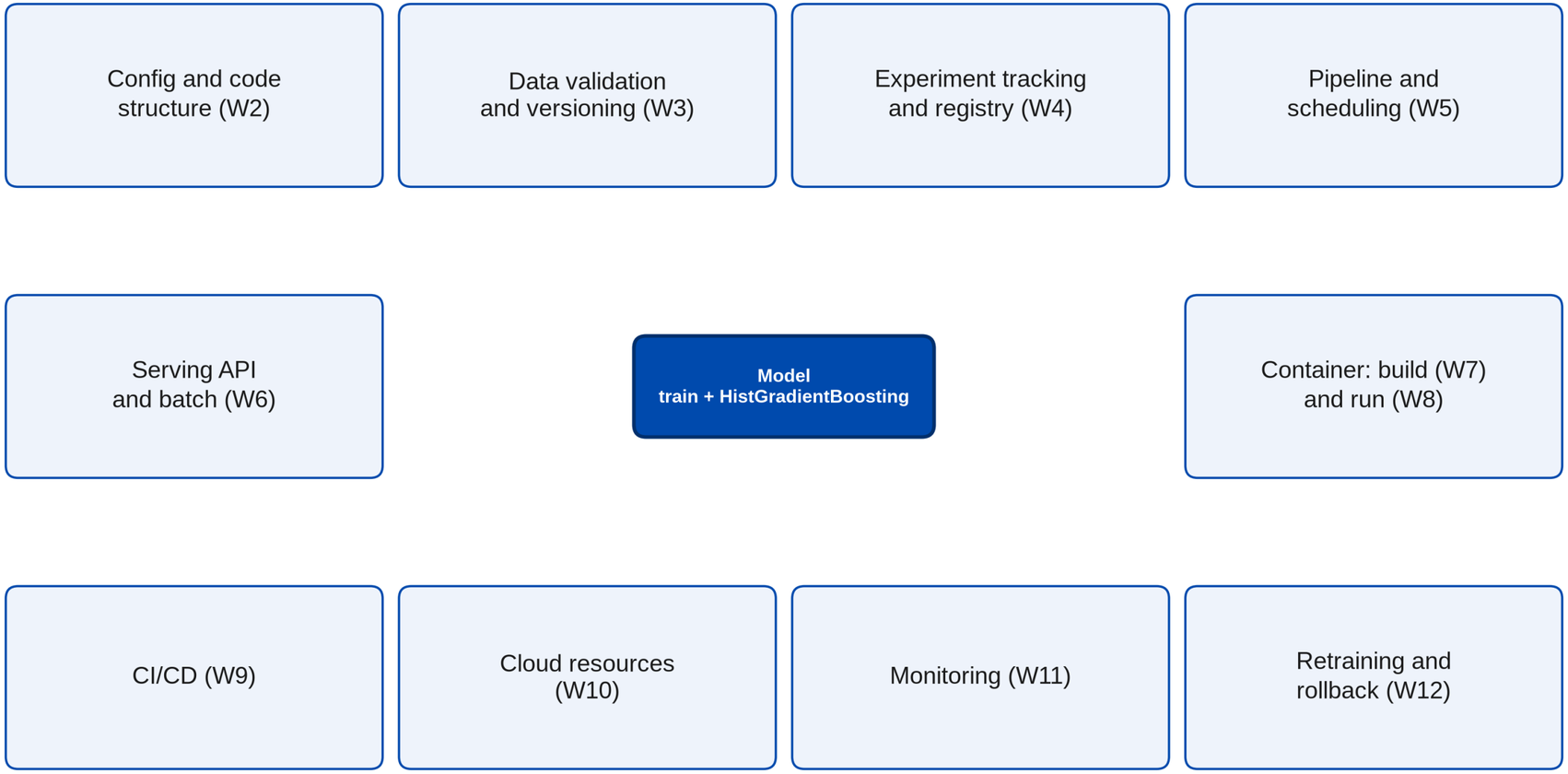
InnovatiCS 2024. All rights reserved

The notebook works, and says what it is not

- MAE **279.29 USD** on 10,755 holdout rows, R2 **0.9826** (tag `m00`)
- **Monolithic and stateful:** cells depend on the order you ran them
- **Hardcoded:** paths and parameters live inside cells
- **No unit tests** of the model code (Week 0 only re-checks the final numbers)
- **No one-command retrain:** nothing trains and saves a model outside the notebook
- Cell 24: the lesson is making this result reproducible, testable and deployable

INNOVATICS
www.innovatics.ai

The model is a small box



Often quoted (2019): about 87% of ML projects never reach production.

MLOps

What it is

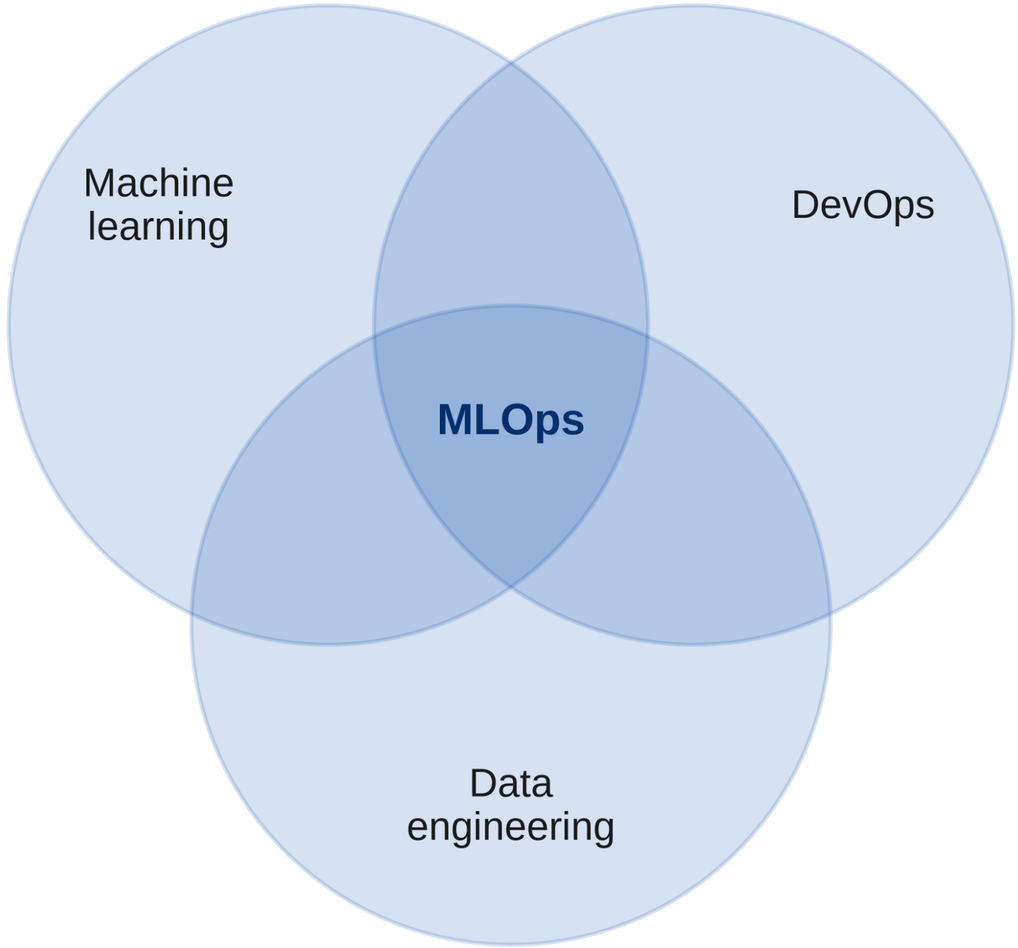
- The engineering practice and team culture that joins building ML systems with running them
- It works step by step toward automating and monitoring every stage of the lifecycle

On the diamond project

- 3 of our 8 risks come before deployment: bad data, untracked data, no reproducibility

INNOVATICS
www.innovatics.ai
Learn Data Science & AI From Zero to Hero

Where MLOps sits



DevOps, DataOps, MLOps

	DevOps	DataOps	MLOps
Main product	Application code	Analytics data, reports	Models and the pipelines that make them
Versioned	Code, config	Code, config, data, environments	Code, config, data, models
Typical failure	Bug, failed build, crash	Late, missing or wrong data	Accuracy falls as the world changes
How you notice	Tests fail, errors, alerts	Data tests, pipeline monitoring	Data checks, drift reports, live error

A model can go wrong with no bug at all

Software

- Loud bugs: a crash, a failed test
- Quiet bugs: a wrong calculation that still returns a plausible number

Models add a third kind

- The code is correct and runs fine
- The world changed, so predictions slowly get worse
- No error is raised. Only measuring against reality shows it

INNOVATICS
www.innovatics.ai
Learn Data Science & AI From Zero to Hero

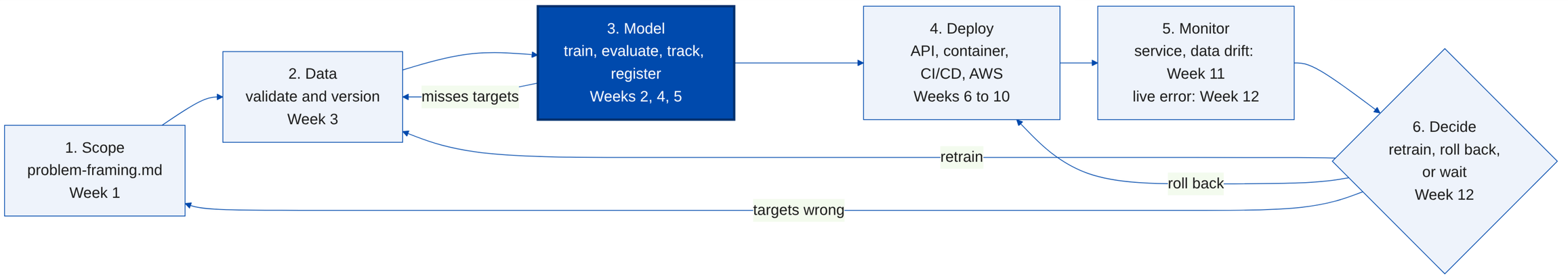
02

Lifecycle and problem framing

INNOVATICS
www.innovatics.ai
Learn Data Science & AI - From Zero to Hero

InnovatiCS 2024. All rights reserved

The diamond lifecycle



Today only stage 3 exists, and only inside a notebook. Every other stage is built in the course.

Each stage answers one question

Stage	Question it answers	Week
1. Scope	Who uses the prediction, and what is "good"?	1
2. Data	Is the data valid, and which version trained the model?	3
3. Model	Is this model better than the current one?	2, 4, 5
4. Deploy	Can other programs use the model, safely and repeatably?	6 to 10
5. Monitor	Is the service healthy, and is the model still right?	11 to 12
6. Decide	The model got worse. What now?	12

Draw it as code with Mermaid

```
flowchart LR
  SCOPE["1. Scope<br/>problem-framing.md<br/>Week 1"]
  DATA["2. Data<br/>validate and version<br/>Week 3"]
  MODEL["3. Model<br/>train, evaluate, track, register<br/>Weeks 2, 4, 5"]
  DEPLOY["4. Deploy<br/>API, container, CI/CD, AWS<br/>Weeks 6 to 10"]
  MONITOR["5. Monitor<br/>service, data drift, live error<br/>Week 11"]
  DECIDE{"6. Decide<br/>retrain, roll back,<br/>or wait<br/>Week 12"}

  SCOPE --> DATA --> MODEL --> DEPLOY --> MONITOR --> DECIDE
  DECIDE -->|retrain| DATA
  DECIDE -->|roll back| DEPLOY
  DECIDE -->|targets wrong| SCOPE
  MODEL -->|misses targets| DATA
```

docs/lifecycle.md (lines 11-23) at tag m01

Note: the file says Week 11 for live error; in the course, live error arrives in Week 12.

Vague goals vs measurable targets

Vague	Measurable (today)
"The price should be accurate"	MAE 300 USD or less on the holdout (279.29)
"It should follow the price pattern"	R2 0.98 or more (0.9826)
"It should be fast"	Online p95 latency 500 ms or less (not measured)
"The daily batch should be quick"	15 minutes or less for 50,000 rows (not measured)
"It should be cheap"	10 USD a month or less on AWS at course traffic
"It should not miss expensive stones"	Report MAE above 10,000 USD with row count. No target yet

Two kinds of targets

```
- **Offline model gate** (Weeks 4, 5, 9, and 12). A new model can become
  `champion` only if, on the same holdout:
  - MAE is 300 USD or less,
  - R2 is 0.98 or more, and
  - MAE is not worse than the current `champion`'s MAE.
- **Release and operating targets** (from Week 6 on). Online p95 latency
  (Week 6, then Weeks 8 and 11), batch run time (Week 6), and cloud cost
  (Week 10) are measured when those stages exist. They are not part of the
  offline gate. Until a target is measured, it is "not measured". It never
  counts as passed.
```

docs/problem-framing.md (lines 44-53) at tag m01

"Not measured" never counts as passed.

INNOVATICS
www.innovatics.ai
Learn Data Science & AI From Zero to Hero

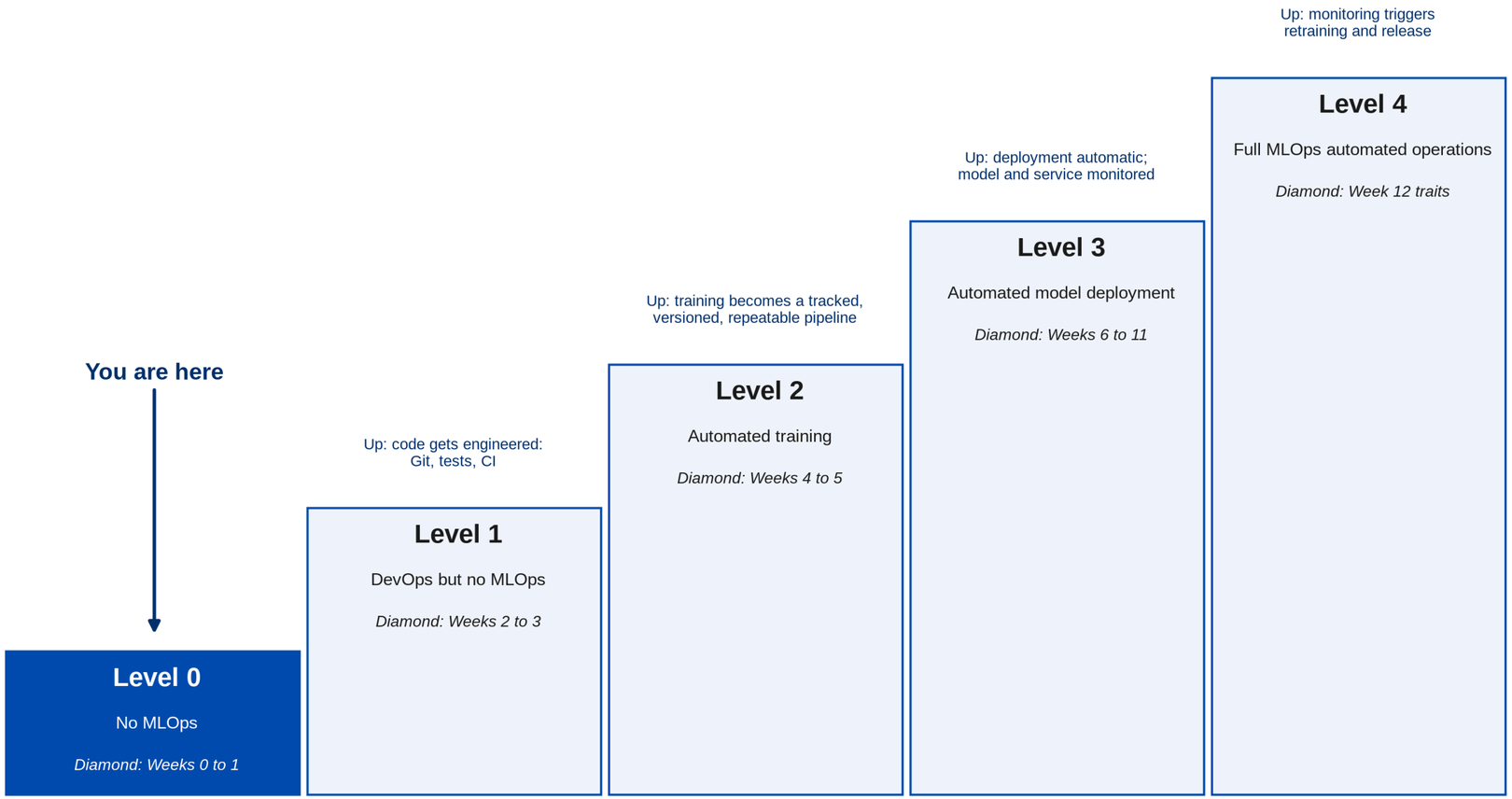
03

The maturity model

www.innovatics.ai
Learn Data Science & AI - From Zero to Hero

InnovatiCS 2024. All rights reserved

Five levels, Level 0 to 4



Levels are cumulative. Real teams are mixed: "mostly Level 1, some Level 2" is a valid score.

The course scale

Level	What is true at this level	Who works together
0 No MLOps	Manual, in notebooks. Model evaluated by hand. No tracking, no automated quality gate	Data scientist alone
1 DevOps but no MLOps	Code in Git, unit and integration tests, CI on every push. Training still manual	Developers help
2 Automated training	Repeatable training pipeline. Data and models versioned. Runs tracked, registry	One team
3 Automated deployment	CD pipeline. Model and service monitored. Often A/B tests	Data, ML, software
4 Full automated operations	Alerts, schedules or code changes trigger retrain and release	Plus production metrics

Not every model needs Level 4

- A proof of concept can stay at Level 0
- Production needs at least Level 1
- Several models make Level 2 worth it
- Level 2 or 3 with monitoring is often enough, especially where a human signs off
- Choosing the target level is part of problem framing

INNOVATICS
www.innovatics.ai

Score a team per pillar

Pillar	Level 0	Level 1	Level 2	Level 3	Level 4
Version everything	☐	☐	☐	☐	☐
Track experiments	☐	☐	☐	☐	☐
Orchestrate	☐	☐	☐	☐	☐
Serve	☐	☐	☐	☐	☐
Containerize	☐	☐	☐	☐	☐
CI/CD	☐	☐	☐	☐	☐
Monitor	☐	☐	☐	☐	☐
Infrastructure as code	☐	☐	☐	☐	☐

Score each pillar on its own. Then one overall score, and the next 3 changes in order.

Where the diamond project climbs

Weeks	What the project gains	Score
0 to 1	One notebook run by hand. No unit tests, no tracking, no deployment	Level 0
2 to 3	Tested package in Git, CI on every push; versioned, validated data	Level 1, one Level 2 trait
4 to 5	Tracked runs, registry with `champion`, a pipeline that trains by itself	Level 2
6 to 10	API, container, CI/CD release to AWS	Level 3 for deployment
11 to 12	Monitoring, retraining, rollback	Level 3 complete, Level 4 traits

04

Hidden technical debt

INNOVATICS
www.innovatics.ai
Learn Data Science & AI - From Zero to Hero

InnovatiCS 2024. All rights reserved

Technical debt in ML

What it is

- Shortcuts that speed you up now and cost you later (Ward Cunningham, 1992)
- ML has normal code debt plus extra debt at the system level

Why normal fixes are not enough

- Data shapes the system's behavior, so module boundaries do not hold
- Refactoring and more unit tests alone do not pay it off

Six kinds of ML debt, and who pays them off

Debt	Diamond example	Paid off in
Glue code	Reshaping written once for the notebook, again for an API	W2: one saved pipeline file, loaded by predict and the API
Pipeline jungles	`clean_v2_final.csv`, cells run in a remembered order	W3 DVC stages, W5 Prefect flow
Data dependencies	Supplier renames "Very Good" to "Very-Good"	W3: versioned data, pandera schema
Feedback loops	Sellers accept the suggestion, so "sale prices" echo the model	W12 measures it, does not remove it
Configuration	Paths and parameters hardcoded in cells (real: cell 27)	W2 `params.yaml` plus a test. Partly paid
External world	"MAE 300 USD" stops fitting if prices inflate	W11 to W12 monitoring; revisit thresholds

Find the debt in our notebook

- **Configuration:** hardcoded paths and parameters (cell 27 lists them)
- **Pipeline jungle:** cleaning steps that must run in a remembered order
- **Data dependency:** a CSV read with no schema
- Which week pays each one off?

INNOVATICS
www.innovatics.ai
Learn Data Science & AI From Zero to Hero

05

Pillars, roles and tools

INNOVATICS
www.innovatics.ai
Learn Data Science & AI - From Zero to Hero

InnovatiCS 2024. All rights reserved

8 pillars across the course

Version everything	Track and register	Orchestrate	Serve
Git and GitHub, DVC, MLflow registry	**MLflow (alias champion)**	**Prefect**	**FastAPI, batch script**
W2 code, W3 data, W4 models	W4	W5	W6

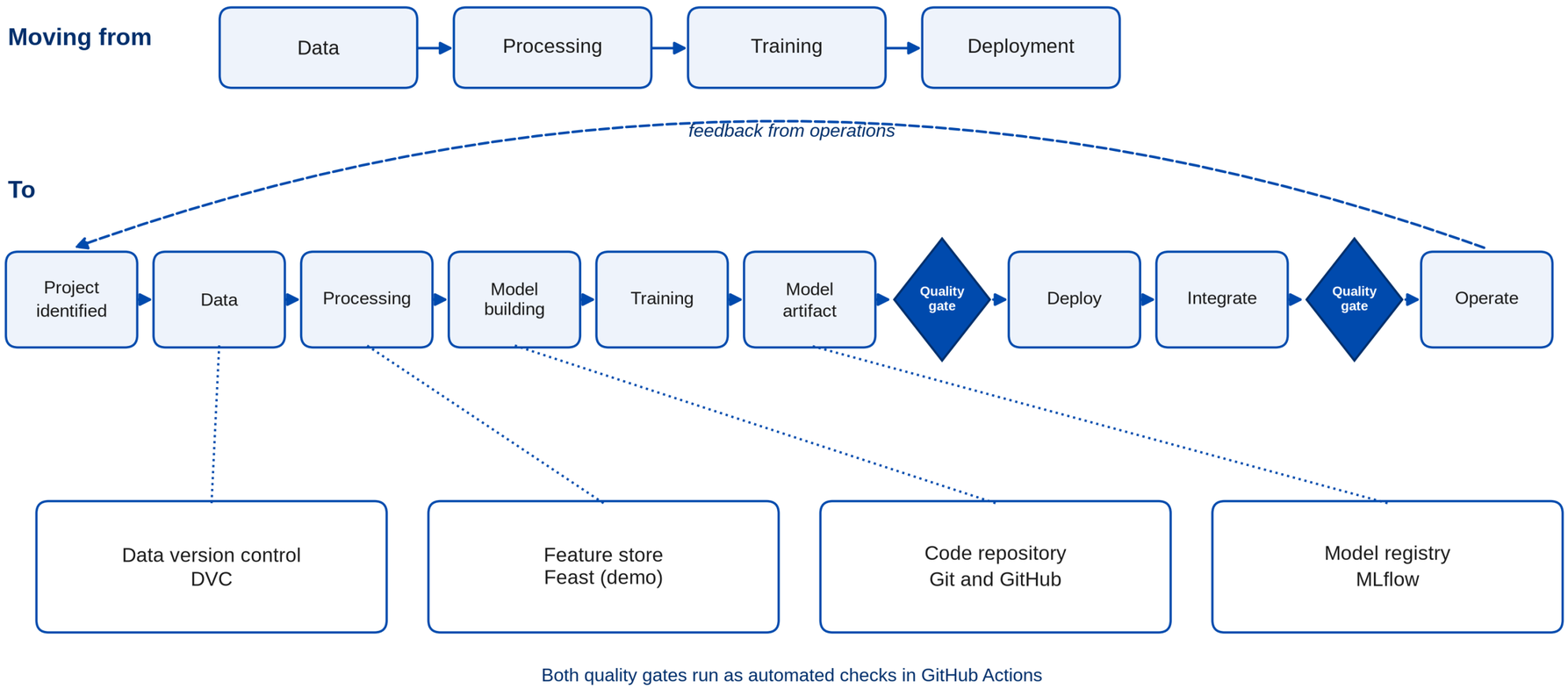
Containerize, then run	CI/CD	Infrastructure as code	Monitor
Docker, Amazon ECR; Kubernetes on Minikube	**GitHub Actions**	**Terraform on AWS**	**Evidently, Prometheus, Grafana**
W7 build and store, W8 run and scale	W2 first check, W9	W10	W11 (W12 retrain, roll back)

The pillars line up with the maturity steps: each one moves a level up.

Coverage check: roadmap.sh MLOps components

roadmap.sh component	Our week
Version control	W2, W3
CI/CD; orchestration	W9; W5
Experiment tracking and model registry	W4
Data lineage and feature stores	Partly: DVC and MLflow (W3, W4); Feast demo (W3)
Model training and serving	W4, W6
Monitoring and observability	W11

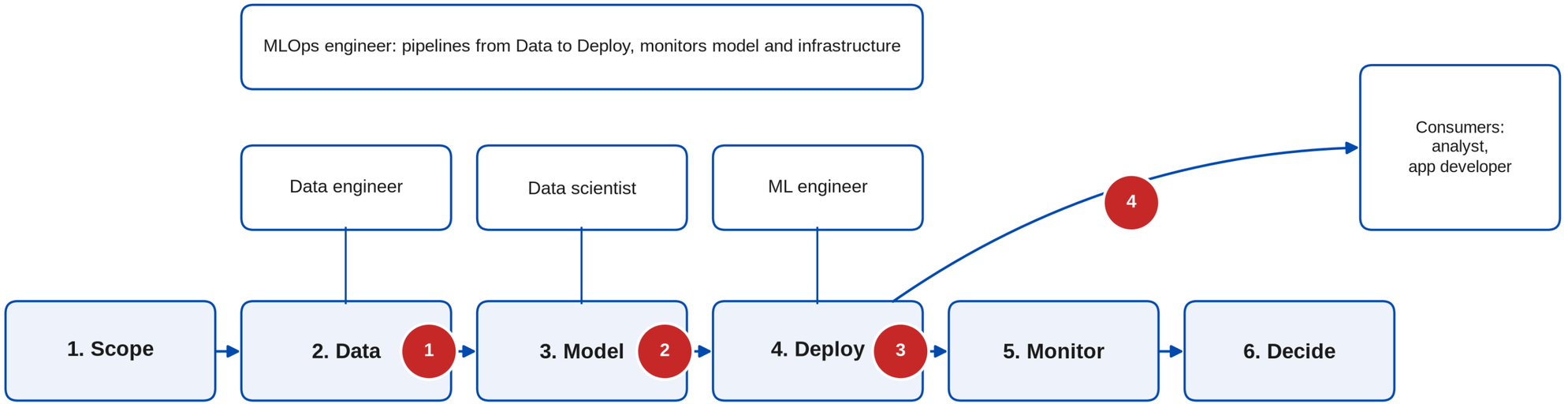
From a workflow to a system



Both quality gates run as automated checks in GitHub Actions

Traceability: every model links back to its data, code and config

Who hands what to whom



1 Data engineer to data scientist: data changes, nobody tells the model owner

2 Data scientist to engineer: notebook re-implemented, training and serving disagree

3 Engineer to operations: nobody owns "is the model still right?"

4 Model to consumers: hidden users break when the model changes

Where hand-offs fail, and the course fix

Hand-off	What goes wrong	Course fix
Data engineer → data scientist	Data changes and nobody tells the model owner	W3: versioned data, schema check
Data scientist → engineer	Notebook re-implemented; training and serving disagree	W2: one package for both
Engineer → operations	Nobody owns "is the model still right?"	W11 monitoring; W12 runbook
Model → consumers	Hidden users break when the model changes	W6 API contract; W10 access control

The tool landscape, grouped by pillar

Version code	Version data	Track and register	Orchestrate
Git, GitHub	**DVC** Git LFS, lakeFS	**MLflow** Weights & Biases	**Prefect** Apache Airflow, Kubeflow Pipelines

Serve	Containerize	Run containers	CI/CD
FastAPI Flask	**Docker,** **Amazon ECR** Docker Hub	**Kubernetes** **on Minikube** Amazon EKS	**GitHub Actions** GitLab CI/CD, Jenkins, AWS CodePipeline

Infrastructure as code	Monitor	Platforms
Terraform AWS CloudFormation	**Evidently, Prometheus,** **Grafana** Amazon CloudWatch	**none: we build** **from parts** Amazon SageMaker AI, Databricks

Ask "which pillar does this tool serve?", then compare it with the course tool.

06

A real case and Practical 1

www.innovatics.ai
Learn Data Science & AI - From Zero to Hero

InnovatiCS 2024. All rights reserved

Aramex: notebooks to production in 8 weeks

What the article shows

- Delivery-time models in Jupyter notebooks
- Partner team took them to production in 8 weeks (2019), runs 24/7
- XGBoost on SageMaker, trained on 6.3 million rows
- API Gateway and Lambda, target under 200 ms, up to 4 million predictions a day
- Retrained regularly, for example for COVID-19

What it does not show

- How releases happen
- Whether runs are tracked
- Whether retraining is automatic
- So: serving in production with regular retraining; not enough evidence to score CI/CD

Practical 1: maturity assessment and problem framing

- **Part A:** score a one-page company scenario per pillar; recommend the next 3 changes
- **Part B:** write `docs/problem-framing.md` for the diamond project (4 questions)
- **Part C:** draw the diamond lifecycle (Mermaid recommended)
- **Done when:** score sheet complete, framing answers all 4, diagram shows all stages
- Start from `start-m01`. Pairs now for 12 minutes, finish on your own

Your capstone: a first look

- You build your own end-to-end ML system by Week 14
- Proposal due end of Week 5. Scope approved in Week 6
- Check-in in Week 10. Build in Week 13
- Verification and demo day in Week 14
- The full brief and rubric come next (20 minutes)

INNOVATICS
www.innovatics.ai
Learn Data Science & AI From Zero to Hero

Quiz 1 and next week

- **Quiz 1 opens now:** MLOps vs DevOps, lifecycle stages, maturity levels, ML debt, roles
- **Read:** Sculley et al. 2015, Sections 1 to 5 (about 5 pages)
- **Next week:** from notebook to a tested Python package with Git, pytest and CI

INNOVATICS
www.innovatics.ai
Learn Data Science & AI From Zero to Hero

INNOVATICS
www.innovatics.ai
Learn Data Science & AI From Zero to Hero

THANK YOU



InnovatiCS 2024. All rights reserved
+1 (678) 209-9780